

Research Paper Summaries

Related Work for the CodeGen RAG Capstone Project

VIT Bhopal AIML PGCP Capstone — Group 39, Batch 26

Mahin Nandipa · Abhinaya Thavishi · Ashu Bagul

July 2026

A Note on Sources and Corrections

This document summarizes all twelve references cited in the project README. Each summary covers what the paper is, its core contribution, and its relevance to this project. While verifying citation details against public sources, two small inaccuracies in the README's reference list were found and are corrected here rather than silently repeated:

CoDocBench: the README lists the lead author as "Pal et al., 2024." The paper's actual lead author is Kunal Suresh Pai, not Pal — likely a transcription slip. The corrected citation is used below.

BIRD / BirdBench: the README lists this as "Li et al., 2024." BIRD was published at NeurIPS 2023, not 2024. The corrected year is used below.

AST retrieval (Jiang et al., 2023): this citation could not be matched with confidence to a specific, verifiable paper by an author named Jiang published in 2023. The closest related work found during research is Sun et al., 2023, "Abstract Syntax Tree for Programming Language Understanding and Representation: How Far Are We?" (arXiv:2312.00413), but the author name does not match, so it is not presented as the source below — that section instead describes the general AST-based retrieval technique this project implements and is transparent about the unresolved citation.

One additional observation: "Codex (Chen et al., 2021)" and "pass@k (Chen et al., 2021)" in the README refer to the same paper — pass@k was introduced in the Codex paper itself, not a separate publication. Both entries are kept as the README lists them, with a note in the pass@k summary below.

1. CodeGen: An Open Large Language Model for Code with Multi-Turn Program Synthesis

Nijkamp, Pang, Hayashi, Tu, Wang, Zhou, Savarese, Xiong, 2022

What it is: A family of open-source autoregressive transformer language models for code, released by Salesforce Research in sizes from 350M to 16B parameters, trained on a mixture of natural-language and multi-programming-language corpora (Python, C, C++, Java, JavaScript, Go).

Core contribution: Introduces multi-turn program synthesis — decomposing a programming task into a sequence of natural-language prompts rather than one monolithic specification — and the Multi-Turn Programming

Benchmark (MTPB) to evaluate it, showing multi-turn decomposition improves synthesis quality over single-turn prompting at comparable model scale.

Relevance to this project: CodeGen-350M-multi is the base model this project fine-tunes and serves. Every checkpoint (Rust LoRA and full fine-tune), every generation endpoint, and the baseline rows in the comparison table trace back to this architecture and pretrained checkpoint.

2. Evaluating Large Language Models Trained on Code

Chen, Tworek, Jun, Yuan, Pinto, Kaplan, Edwards, Burda, Joseph, Brockman, Ray, Puri, Krueger, Petrov, Khlaaf, Sastry, Mishkin, Chan, Gray, Ryder, Pavlov, Power, Kaiser, Bavarian, Winter, Tillet, Such, Cummings, Plappert, Chantzis, Barnes, Herbert-Voss, Guss, Nichol, Paine, Tezak, Tang, Babuschkin, Balaji, Jain, Saunders, Hesse, Carr, Leike, Achiam, Misra, Morikawa, Radford, Knight, Brundage, Murati, Mayer, Welinder, McGrew, Amodei, McCandlish, Sutskever, Zaremba, 2021

What it is: The OpenAI paper introducing Codex, a GPT-3 model fine-tuned on public GitHub code, and the model that powers GitHub Copilot.

Core contribution: Introduces the HumanEval benchmark (164 hand-written programming problems with unit tests) and the pass@k metric — an unbiased statistical estimator of the probability that at least one of k sampled generations passes all unit tests — which became the standard way to evaluate functional correctness of generated code.

Relevance to this project: This project reports pass@k as one of its core evaluation metrics for program synthesis, following the same unbiased-estimator formulation defined here.

3. CodeBERT: A Pre-Trained Model for Programming and Natural Languages

Feng, Guo, Tang, Duan, Feng, Gong, Shou, Qin, Liu, Jiang, Zhou, 2020

What it is: A bimodal pre-trained transformer from Microsoft Research, built on the RoBERTa architecture, trained jointly on natural-language and programming-language pairs (docstring/code) across six languages.

Core contribution: Combines masked language modeling with a replaced-token-detection objective (distinguishing generator-produced tokens from original ones) to learn representations that capture semantic correspondence between code and natural-language descriptions, without requiring bimodal data at every training step.

Relevance to this project: CodeBERT-style embeddings and evaluation are foundational to two things this project uses directly: CodeBERTScore (paper 11 below), and, more generally, the encoder-based semantic similarity approach that motivates using dense embeddings for retrieval in the RAG pipeline alongside AST-based structural retrieval.

4. CoDocBench: A Dataset for Code-Documentation Alignment in Software Maintenance

Pai, Devanbu, Islam, and coauthors, 2024

What it is: A benchmark dataset built for evaluating how well models keep code and its accompanying documentation (docstrings, comments) consistent as code changes over time, drawn from real-world open-source repository history.

Core contribution: Provides paired before/after code-and-documentation snapshots from real commits, enabling evaluation of models on the task of detecting and generating documentation updates that stay faithful to underlying code changes, rather than treating documentation generation as a one-shot, static task.

Relevance to this project: Directly informs this project's documentation-generation task and its evaluation methodology: producing accurate, up-to-date docstrings/comments for a given code snippet, scored here with CodeBLEU and BERTScore against reference documentation.

Note: The README cites this as "Pal et al., 2024"; the paper's actual lead author is Kunal Suresh Pai. Corrected here.

5. Spider: A Large-Scale Human-Labeled Dataset for Complex and Cross-Domain Semantic Parsing and Text-to-SQL Task

Yu, Zhang, Yang, Yasunaga, Wang, Li, Ma, Li, Yao, Roman, Zhang, Radev, 2018

What it is: A large, cross-domain, human-annotated text-to-SQL benchmark from Yale University spanning 200 databases across 138 domains, with complex SQL queries involving joins, nested queries, and set operations.

Core contribution: Unlike earlier single-database text-to-SQL datasets, Spider requires models to generalize to unseen databases and schemas at test time, making it a much harder and more realistic test of semantic parsing than prior benchmarks, and establishing exact-set-match and execution accuracy as standard evaluation metrics.

Relevance to this project: Spider is one of the two text-to-SQL benchmarks this project evaluates against (sql_generation_spider in the comparison table), testing the system's ability to translate natural-language questions into correct SQL over previously unseen database schemas.

6. Can LLM Already Serve as A Database Interface? A Big Bench for Large-Scale Database Grounded Text-to-SQLs (BIRD)

Li, Hui, Qu, Yang, Li, Li, Wang, Qin, Geng, Huo, Zhou, Ma, Li, Huang, 2023

What it is: A large-scale text-to-SQL benchmark (BIRD/BirdBench) built around real, messy, large production databases rather than the cleaner academic schemas used in earlier benchmarks like Spider, and explicitly designed to test external-knowledge reasoning and query efficiency.

Core contribution: Highlights and measures gaps that Spider-style benchmarks miss: dirty/noisy real-world data values, the need for external domain knowledge to interpret ambiguous questions, and SQL execution efficiency on large tables — showing that strong Spider performance does not guarantee strong performance under these more realistic conditions.

Relevance to this project: BIRD is this project's second text-to-SQL evaluation set (sql_generation_birdbench), used alongside Spider to test robustness on harder, more realistic databases; the comparison table's lower execution-accuracy score on BIRD versus Spider is consistent with BIRD's documented difficulty.

Note: The README cites this as "Li et al., 2024"; BIRD was published at NeurIPS 2023. Corrected here.

7. Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks

Lewis, Perez, Piktus, Petroni, Karpukhin, Goyal, Küttler, Lewis, Yih, Rocktäschel, Riedel, Kiela, 2020

What it is: The Facebook AI Research paper that introduced Retrieval-Augmented Generation (RAG) as a general architecture: a sequence-to-sequence generator conditioned on documents retrieved from a non-parametric memory (a dense vector index over Wikipedia), rather than relying solely on knowledge stored in model weights.

Core contribution: Shows that combining a learned dense retriever with a generator, trained end-to-end, outperforms purely parametric models on knowledge-intensive tasks, and that the retrieved-document memory can be swapped or updated at inference time without retraining the generator — decoupling factual knowledge from model parameters.

Relevance to this project: This is the architectural blueprint for the project's entire RAG subsystem: retrieving relevant code/context (via dense FAISS search and AST-based structural search, fused with reciprocal rank fusion) and packing it into the prompt before generation, rather than depending only on what the base CodeGen model memorized during pretraining.

8. ReACC: A Retrieval-Augmented Code Completion Framework

Lu, Duan, Han, Guo, Hu, Svyatkovskiy, 2021

What it is: A Microsoft Research framework for code completion that combines a retrieval stage (finding similar code snippets via both lexical/sparse and dense/GraphCodeBERT-based search) with a generation stage conditioned on the retrieved snippets.

Core contribution: Demonstrates that augmenting a code-completion generator with retrieved, semantically similar code from a large corpus meaningfully improves completion accuracy over generation-only baselines, and that combining sparse and dense retrieval signals outperforms either alone.

Relevance to this project: ReACC is the closest prior work to this project's core hybrid-retrieval design: it validates the same general strategy of fusing sparse/lexical and dense retrieval signals before generation, which this project implements as reciprocal rank fusion between dense (FAISS) and AST-structural retrieval.

9. AST-Based Retrieval for Code (cited in the README as Jiang et al., 2023)

Citation unverified — see note, 2023

What it is: The README cites this as motivation for using abstract syntax tree (AST) structure, rather than only surface text or token embeddings, as a retrieval signal for finding structurally similar code.

Core contribution: The general technique — parsing code into its AST and using tree structure (or tree-edit-distance / subtree matching) as a similarity signal — is well established in the code-retrieval literature; this project's own AST index and search over parsed syntax trees follows that general approach.

Relevance to this project: The project's AST-based retrieval index (fused with dense retrieval via reciprocal rank fusion) is a direct application of AST-structural similarity search as a complement to embedding-based dense retrieval.

Note: No publication by an author named Jiang from 2023 could be verified with confidence as matching this citation. The closest related paper found is Sun et al., 2023, "Abstract Syntax Tree for Programming Language Understanding and Representation: How Far Are We?" (arXiv:2312.00413) — but the author name does not match, so it is not asserted as the source. This citation should be double-checked against the original proposal or literature notes before the report is finalized or submitted externally.

10. CodeBLEU: a Method for Automatic Evaluation of Code Synthesis

Ren, Guo, Lu, Zhou, Blanco, Ma, 2020

What it is: A code-specific evaluation metric proposed by Microsoft Research and Sun Yat-sen University, extending BLEU (a text n-gram overlap metric) to account for code's structural and semantic properties.

Core contribution: Defines CodeBLEU as a weighted combination of four signals: standard n-gram match (BLEU), weighted n-gram match (giving more weight to language keywords), syntactic AST match, and semantic dataflow match — showing this combination correlates far better with human judgments of code quality than plain BLEU or exact match.

Relevance to this project: CodeBLEU is a primary metric reported in this project's comparison table for every generation task. This session's work specifically closed the gap between the metric's design and its implementation:

adding tree-sitter-python so the AST-match and dataflow-match components are computed from real parsed syntax trees instead of silently falling back to a plain sacrebleu approximation.

11. CodeBERTScore: Evaluating Code Generation with Pretrained Models of Code

Zhou, Yasunaga, Yang, Guha, Radev, Neubig, 2023

What it is: An evaluation metric that adapts BERTScore (soft token-matching using contextual embeddings, rather than exact n-gram overlap) to source code, using pretrained code-language-model embeddings and incorporating surrounding code context.

Core contribution: Shows that soft, embedding-based similarity between generated and reference code correlates more strongly with human judgments and functional correctness than n-gram-based metrics like BLEU or CodeBLEU alone, especially for capturing semantically equivalent code written with different variable names or minor structural differences.

Relevance to this project: Reported as `bertscore_f1` in this project's comparison table for program synthesis, commit-message generation, and code translation — used alongside CodeBLEU and exact-match so that surface-level rewording differences don't unfairly penalize semantically correct generations.

12. pass@k (introduced in Chen et al., 2021 — see entry 2)

Chen, Tworek, Jun, Yuan, and coauthors (OpenAI), 2021

What it is: An unbiased statistical estimator for the probability that at least one of k independently sampled code completions for a given problem passes all provided unit tests, computed without the high variance of naively sampling only k completions per problem.

Core contribution: Formalizes $\text{pass}@k$ as $1 - C(n-c, k) / C(n, k)$ where n completions are sampled and c of them pass, giving a metric that is comparable across different sampling budgets and doesn't require re-sampling exactly k completions to estimate $\text{pass}@k$ for any $k \leq n$.

Relevance to this project: Used by this project as one of the functional-correctness metrics for program synthesis, reported alongside CodeBLEU and BERTScore in the comparison table.

Note: The README lists this as a separate reference from Codex (entry 2), but $\text{pass}@k$ was introduced within the Codex paper itself rather than a distinct publication — both entries describe the same underlying source.

Summary

Together these twelve references trace the project's full lineage: the base language model and its evaluation methodology (CodeGen, Codex/pass@k, CodeBERT), the SQL-generation benchmarks used for the text-to-SQL task (Spider, BIRD), the retrieval-augmented-generation architecture and its closest code-specific precedent (RAG, ReACC, AST retrieval), the documentation-alignment task this project also evaluates (CoDocBench), and the automatic metrics used to score every generation task (CodeBLEU, CodeBERTScore).

The two citation corrections (CoDocBench's author name, BIRD's publication year) and the one unresolved citation (AST retrieval) are flagged above rather than left silent, so the team can correct the README and, if this document accompanies a submission, cite the AST-retrieval work accurately once confirmed.